AI-TECH

Przykład 2 DO LABORATORIUM 2

z UCZENIA MASZYNOWEGO

Klasyfikacja nadzorowana

Szymon Zaporowski

Klasyfikacja z użyciem Drzew Decyzyjnych i algorytmu Lasu Losowego - przykłady

Projekt współfinansowany ze środków Unii Europejskiej w ramach Europejskiego Funduszu Rozwoju Regionalnego Program Operacyjny Polska Cyfrowa na lata 2014-2020, Oś Priorytetowa nr 3 "Cyfrowe kompetencje społeczeństwa" Działanie nr 3.2 "Innowacyjne rozwiązania na rzecz aktywizacji cyfrowej" Tytuł projektu: „Akademia Innowacyjnych Zastosowań Technologii Cyfrowych (AI Tech)”

1. Przykład 2

Drzewa Decyzyjne (ang, Decission Trees) są wszechstronnymi algorytmami uczenia maszynowego, które mogą być wykorzystywane zarówno jako klasyfikatory jak i do zadań regresji. Można je wykorzystać do tworzenia zaawansowanych modeli zdolnych do wykonywania zadań ze złożonymi zbiorami danych. Drzewa decyzyjne są składowymi elementami algorytmu losowych lasów (ang. Random Forest)

In [30]:
```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import make_classification
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

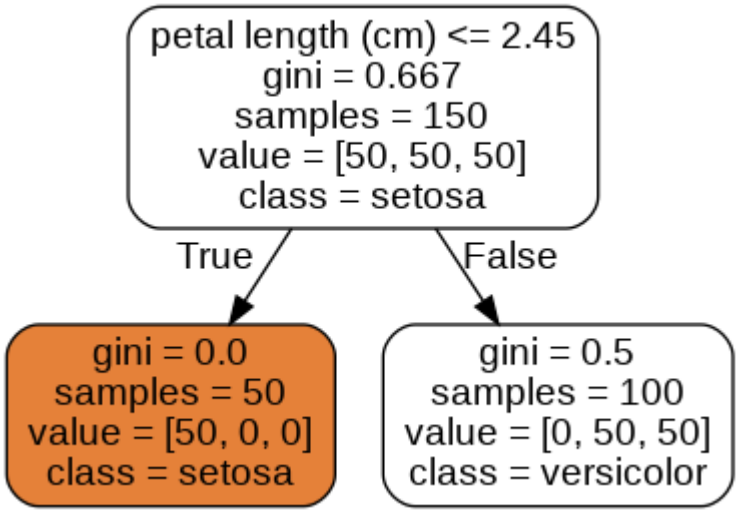
Do przedstawienia idei stojącej za mechanizmem działania algorytmu drzewa decyzyjnego wykorzystamy prosty przykład z użyciem datasetu iris. W tym przykładzie skupimy się na parametrze max_depth oznaczającego maksymalne zagłebienie drzewa.

```
tree_clf = DecisionTreeClassifier(max_depth=1)
```

Zobaczmy jaki będzie wynik wykonania takiego kodu. Przy użyciu modułu export_graphviz oraz pydot tworzymy graficzną reprezentację drzewa, która zostaje zapisana na dysku zgodnie z podaną nazwą.

In [31]:
```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

iris = load_iris()
X= iris.data[:,2:]
y = iris.target

tree_clf = DecisionTreeClassifier(max_depth=3)
tree_clf.fit(X,y)

from sklearn.tree import export_graphviz
import pydot
export_graphviz(
    tree_clf,
    out_file='drzewo.dot',
    feature_names=iris.feature_names[2:],
    class_names=iris.target_names,
    rounded = True,
    filled=True
)
(graph, ) = pydot.graph_from_dot_file('drzewo.dot')# Zapis do pliku png
graph.write_png('drzewo.png')
```

Zobaczmy jak wygląda drzewo po wykonaniu kodu:



W głównym weźlę czyli korzeniu na wysokości 0 zostało zadane pytanie - czy długość płatka jest mniejsza niż 2,45 cm. Jeśli warunek jest spełniony znajdujemy się w liściu na wysokości 1. Drzewo przewiduje iż znajduje się to klasa setosa.

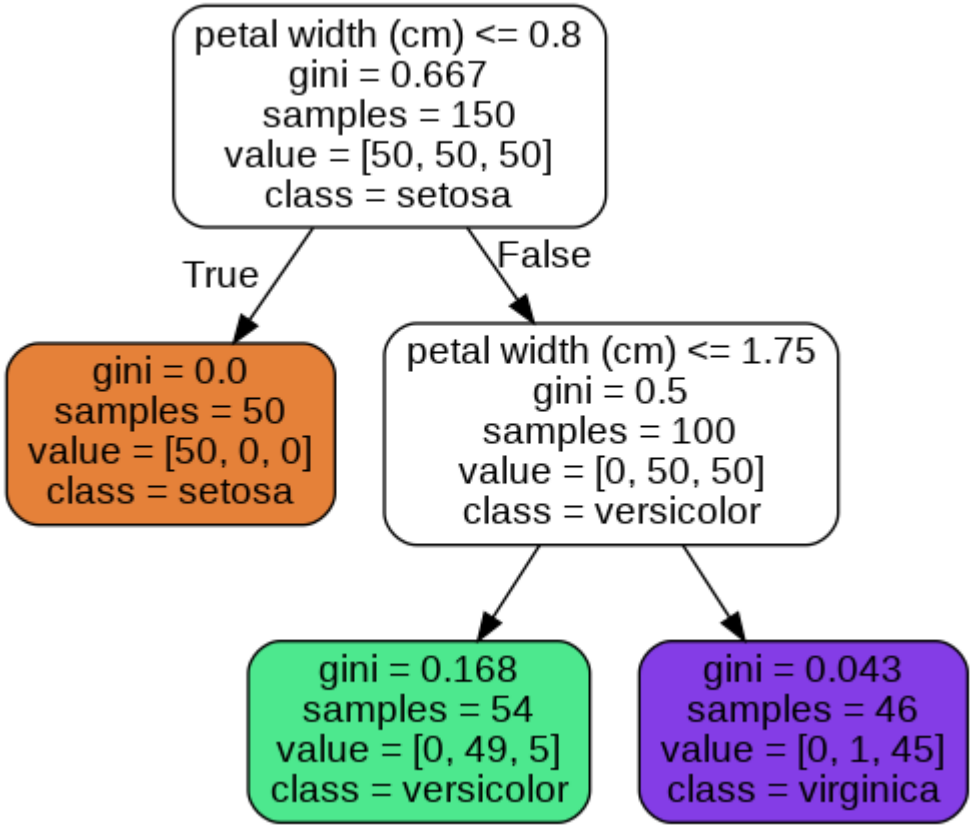
A co w przypadku, gdfy warunke nie zostaje spełiony i znajdujemy się w prawym liściu? Wskazana klasa to versicolor.

Teraz pozostaje zrozumieć czym są wszystkie pozostałe atrybuty znajdujące się w drzewie.

> Atrybut *sample* odpowiada za liczbę próbek uczących wyznaczonych dla danego węzła.
>
> Atrubut *value* wskazuje jak wiele przykładów uczących z każdej klasy należy do danego węzła lub liścia. W przypadku liścia po lewej stronie mamy 50 próbek należących do klasy *setosa* oraz po 0 próbek należących do pozostałych klas. W przypadku prawego liścia jest 50 próbek należacych do klasy *versicolor* i 50 należacych do klasy *virginica*.

Widać teraz, że mamy tylko 2 liście i aż 3 klasy. Do tego jeden z liści zawiera po tyle samo próbek dwóch klas. Oznacza to, iż drzewo jest za płytkie i poziom zagłębienia oznaczany parametrem *max_depth* powinien zostać zwiększony.

Zanim do tego przejdziemy pozostaje do rozszyfrowania ostatni parametr: *gini* - jest on rozumiany jako miara zanieczyszczenia. Oznacza to, iż będzie wynosił 0 jeśli wszystkie próbki znajdujące się w danym węźle czy liściu będą przynależeć tylko do jednej klasy. Dobrym przykładem jest tutaj lewy liść dla którego wartość parametru *gini* jest równa 0.



Widzimy, że zmieniło się pytanie zadawane w korzeniu oraz pojawiło się pytanie węźle na poziomie 1 oraz dwa dodatkowe liście wychodzące z tego węzła. Zmieniły się również wartości atrybutów, zarówno *samples* jak i *gini*.

W jaki sposób szacować prawdopodobieństwo przynależności próbki do danej klasy dla drzew decyzyjnych?

Wystarczy użyć `.predict.proba()` podając jako agrumenty wartości danej próbki. W wyniki otrzymane zostaną prawdopodobieństwa przynależności do klas występujących w danym drzewie.

Z wykorzystaniem polecenia `predict()` i podając wartość otrzyma się konkretną przynależnośc do klasy. Wystarczy odpalić poniższą komórkę i zobaczyć jakie zostaną otrzymane wyniki.

In [32]:
```
print(tree_clf.predict_proba([[8,0.5]]))
tree_clf.predict([[8,0.5]])
```

```
[[0.         0.33333333 0.66666667]]
```

Out[32]:
```
array([2])
```

Teraz spróbujemy wykonać takie same operacje dotyczace tworzenia klasyfikatora, ale z wykorzystaniem algorytmu Random Forest.

Do tego celu wykorzystamy funkcję:

```
RandomForestClassifier(max_depth=2, random_state=0)
```

Więcej informacji dotyczących parametrów tej funkcji można znaleźć na stronie: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html?highlight=random%20forest#sklearn.ensemble.RandomForestClassifier

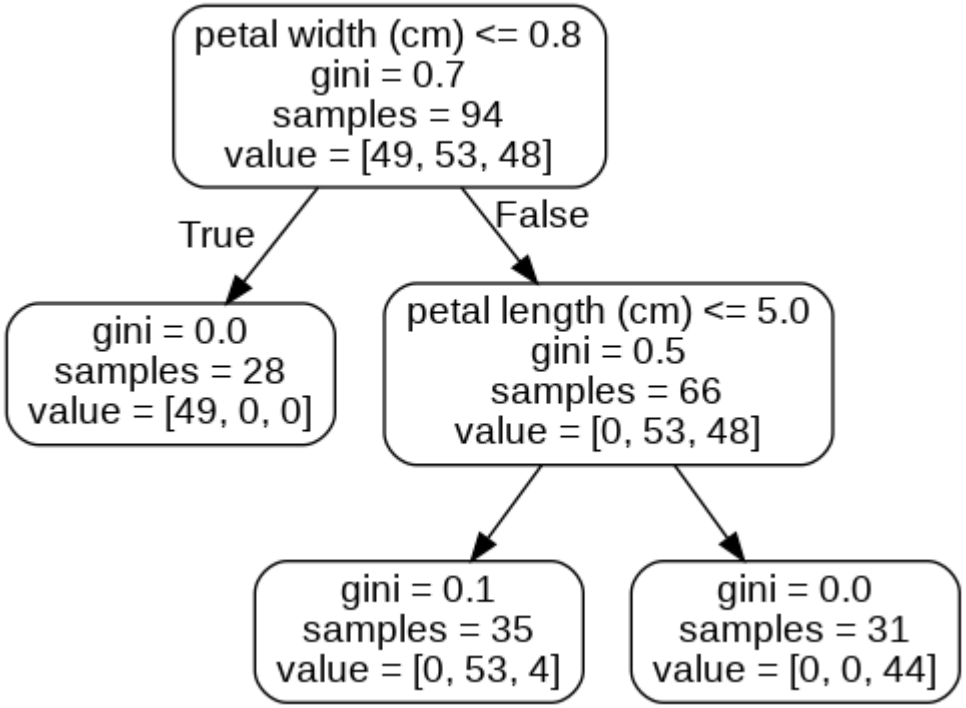
In [33]:
```
dataset = datasets.load_iris()
X = dataset.data
y = dataset.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.10,random_state=109)


clf = RandomForestClassifier(max_depth=2, random_state=0)
clf.fit(X, y)

#print(clf.score)
#print(clf.predict([[0, 0, 0, 0]]))

tree = clf .estimators_[5]
export_graphviz(tree, out_file = 'tree.dot', feature_names = dataset.feature_names, rounded = True, precision = 1)# Use dot file to create
(graph, ) = pydot.graph_from_dot_file('tree.dot')# Write graph to a png file
graph.write_png('tree.png')

y_pred= clf.predict(X_test)
print("Wektor etykiet dla danych testowych:",y_test)
print("Wektor etykiet wytypowany przez klasyfikator:",y_pred)
print('Accuracy:')
accuracy_score(y_test, y_pred)
```

```
Wektor etykiet dla danych testowych: [2 1 2 0 2 1 0 2 1 2 2 0 1 0 0]
Wektor etykiet wytypowany przez klasyfikator: [2 1 2 0 2 1 0 2 1 2 2 0 1 0 0]
Accuracy:
```

Out[33]:
```
1.0
```

Jak widać wektory zgadzają się ze sobą, a jak wygląd graficzna reprezentacja grafu takiego drzewa? Wystarczy spojrzeć poniżej aby się przekonać.

Jak widać nie zmieniło się wiele w prównaniu z algorytmem drzewa decyzyjnego.



Skoro mamy za sobą wprowadzenie do drzew decyzyjnych, przejdziemy do konkretniejszych przykładów. W tym celu wykorzystamy dataset diabetes, który jest udostępniony na stronie Kaggle na licencji C0.

In [34]:
```
import pandas as pd

features = pd.read_csv('diabetes.csv')
features.head(5)
```

Out[34]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Wyświtlamy kształt naszego datasetu, mamy 8 cech i etykietę na końcu,w sumie drugi wymiar to 9.

In [35]:
```
print('The shape of our features is:', features.shape)
print('our features is:', features)
```

```
The shape of our features is: (768, 9)
our features is:      Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0                 6      148             72             35        0  33.6
1                 1       85             66             29        0  26.6
2                 8      183             64              0        0  23.3
3                 1       89             66             23       94  28.1
4                 0      137             40             35      168  43.1
..              ...      ...            ...            ...      ...   ...
763              10      101             76             48      180  32.9
764               2      122             70             27        0  36.8
765               5      121             72             23      112  26.2
766               1      126             60              0        0  30.1
767               1       93             70             31        0  30.4

     DiabetesPedigreeFunction  Age  Outcome
0                       0.627   50        1
1                       0.351   31        0
2                       0.672   32        1
3                       0.167   21        0
4                       2.288   33        1
..                        ...  ...      ...
763                     0.171   63        0
764                     0.340   27        0
765                     0.245   30        0
766                     0.349   47        1
767                     0.315   23        0

[768 rows x 9 columns]
```

Testowo sprawdzamy statystykę danych z użyciem funkcji:

```
#features.describe()
```

In [36]:
```
features.describe()
```

Out[36]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	0.348958
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000

Dokonujemy przekształceń z wykorzystaniem biblioteki numpy, musimy rozdzelić etykiety od danych.

In [37]:
```
# Wykorzystujemy bibliotekę numpy aby zamienić dane na tablice numpy
import numpy as np#
labels = np.array(features['Outcome'])# Usuwamy etykiety ze zbioru parametrów
# axis 1 odnosi sie do kolumn
features = features.drop('Outcome', axis = 1)#  Zostawiamy nazwy etykiet do wykorzystania później
feature_list = list(features.columns)# Właściwa konwersja na tablice numpy
features = np.array(features)
```

Standardowy podział danych z wykorzystaniem funkcji:

```
train_test_split()
```

In [38]:
```
from sklearn.model_selection import train_test_split# Podział danych
train_features, test_features, train_labels, test_labels = train_test_split(features, labels, test_size = 0.25, random_state = 42)
```

Sprawdzmay jaki kształt mają dane po podziale, czy wartości dla etykiet zgadzają się z danymi.

In [39]:
```
print('Kształt danych treningowych:', train_features.shape)
print('Kształt etykiet trenignowych:', train_labels.shape)
print('Kształt danych testowych:', test_features.shape)
print('Kształt etykiet testowych:', test_labels.shape)
```

```
Kształt danych treningowych: (576, 8)
Kształt etykiet trenignowych: (576,)
Kształt danych testowych: (192, 8)
Kształt etykiet testowych: (192,)
```

Dokonujemy właściwego importu modelu oraz trenignu.

In [40]:
```
# Import wykorzystywanego modelu
from sklearn.ensemble import RandomForestClassifier# Wykorzystujemy algorytm Random Rofest
rf = RandomForestClassifier(n_estimators = 100, random_state = 42)
rf.fit(train_features, train_labels);
predictions_forest = rf.predict(test_features)
print('Accuracy:')
accuracy_score(test_labels, predictions_forest)
```

```
Accuracy:
```

Out[40]:
```
0.734375
```

Wynik accuracy nie jest najlepszy, ale w tej chwili nie jest to najistoniejsze. Tworzymy graf wytrenowanego klasyfikatora. Najelpiej podejrzeć go otwierając po lewej stronie okno z danymi i pobierając plik randomtree.png

In [41]:
```
# Import bibliotek niezbędnych do wizualizacji
from sklearn.tree import export_graphviz
import pydot
tree = rf.estimators_[5]# Wybór jednego z drzew z lasu
export_graphviz(tree, out_file = 'randomtree.dot', feature_names = feature_list, rounded = True, precision = 1)# Tworzymy graf z wykorzystan
(graph, ) = pydot.graph_from_dot_file('randomtree.dot')# Zapis do pliku png
graph.write_png('randomtree.png')
```

Ze względu na wielkość otrzymanego drzewa dokonujemy zmniejszenia modelu. Powtarzamy czynności dla pliku small_tree.png

In [42]:
```
# Zmniejszamy rozmiar drzewa do maksymalnej złożoności 3 poziomów
rf_small = RandomForestClassifier(n_estimators=10, max_depth = 3)
rf_small.fit(train_features, train_labels)# Extract the small tree
tree_small = rf_small.estimators_[5]# Save the tree as a png image
export_graphviz(tree_small, out_file = 'small_tree.dot', feature_names = feature_list, rounded = True, precision = 1)
(graph, ) = pydot.graph_from_dot_file('small_tree.dot')
graph.write_png('small_tree.png');
```

Algortym losowych lasów posiada bardzo ciekawą właściwość. Z jego wykorzystaniemj można w łatwy sposób dokonać pomiaru względnej istotności każdej z cech zbioru. W przypadku modułu scikit-learn pomiar istotności to miara w jakim stopniu węzły we wszystkich drzewach w lesie zmniejszają zanieczyszczenie korzystając z tej cechy (zmiana atrybutu *gini*). Gdyby spojrzeć na to z matematycznego punktu widzenia jest to nic innego jak średnia ważona przy założeniu,

że waga każdego z węzłów jest równa liczbie powiązanych z danym węzłem próbek treningowych. Bardzo przydatną cechą w module scikit-learn jest fakt, iż automatycznie obliczany jest wynik dla każdej cechy po wykonaniu treningu i dodatkowo jest on skalowany przez co suma wszystkich istotności wynosi 1.

Wykorzystamy tę funkcję drzew losowych do obliczenia najistotniejszych cech w zbiorze.

In [43]:
```
# Liczymy istotność parametrów z wykorzystaniem algorytmu losowych lasów
importances = list(rf.feature_importances_)# Lista krotek ze zmiennymi oraz istotnościami
feature_importances = [(feature, round(importance, 2)) for feature, importance in zip(feature_list, importances)]#  Sortowanie istotności c
feature_importances = sorted(feature_importances, key = lambda x: x[1], reverse = True)# Wypisanie najistotniejszych cech wraz z istotności
[print('Variable: {:30} Importance: {}'.format(*pair)) for pair in feature_importances];
```

```
Variable: Glucose                      Importance: 0.28
Variable: BMI                          Importance: 0.16
Variable: Age                          Importance: 0.14
Variable: DiabetesPedigreeFunction     Importance: 0.12
Variable: BloodPressure                Importance: 0.09
Variable: Pregnancies                  Importance: 0.07
Variable: SkinThickness                Importance: 0.07
Variable: Insulin                      Importance: 0.07
```

Jak widać najistotniejszą cechą jest wartość glukozy, co w sumie nie powinno dziwić w przypadku klsyfikowania cukrzycy. Pozostałe cechy nie odbiegają od siebie istotnością w tak znaczący sposób.

Można teraz sobie zadać pytanie, co by było gdyby dokonać treningu wykorzystując tylko najisotniejsze cech? Czy wyniki będą lepsze czy gorsze? A może czas treningu ulegnie zmianie?

In [44]:
```
# Tworzymy nową instancję algorytmu losowego lasu wykorzystując tylko najistoniejsze cechy

rf_most_important = RandomForestClassifier(n_estimators= 100, random_state=42)# Extract the two most important features
important_indices = [feature_list.index('Glucose'), feature_list.index('BMI')] # wybieramy tylko 2 najistotnijeszcze cechy - glukozę i BMI
train_important = train_features[:, important_indices]
test_important = test_features[:, important_indices]# Trening
rf_most_important.fit(train_important, train_labels)# Predykcja wyników
predictions = rf_most_important.predict(test_important)
print('Accuracy:')
accuracy_score(test_labels, predictions)
```

```
Accuracy:
```

Out[44]:
```
0.7291666666666666
```

Jak widać wartość miary accuracy pełnego modeli i trenowanego tylko z użyciem dwóch najistotniejszych cech nie odbiega od siebie znacząco. Oznacza to, iż lepiej korzystać z modelu, który jest prostszy obliczeniowo.

Zobaczmy jeszcze jak wyglądają pozostałe metryki, w tym celu skorzystajmy z funkcji:

```
# classification_report()
```

Ta funkcja wylicza wartośći miar f1, precison i recall dla poszczególnych klas oraz podaje liczbę sklasyfikownaych przypadków w postaci pola *support*. Dodatkow wyliczane są średnie wartośći macro czyli uśrednianie średniej niezbalandsowanej dla każdej z etykiet oraz średniej ważonej, która uwzględnia ilość danych dla każdej z klas.

In [45]:
```
classification_report(test_labels, predictions, output_dict= True, target_names=[0,1])
```

Out[45]:
```
{0: {'precision': 0.7795275590551181,
  'recall': 0.8048780487804879,
  'f1-score': 0.792,
  'support': 123.0},
 1: {'precision': 0.6307692307692307,
  'recall': 0.5942028985507246,
  'f1-score': 0.6119402985074627,
  'support': 69.0},
 'accuracy': 0.7291666666666666,
 'macro avg': {'precision': 0.7051483949121744,
  'recall': 0.6995404736656062,
  'f1-score': 0.7019701492537314,
  'support': 192.0},
 'weighted avg': {'precision': 0.7260675348273774,
  'recall': 0.7291666666666666,
  'f1-score': 0.7272910447761195,
  'support': 192.0}}
```

Dla pewności można wykonać obliczenie macierzy pomyłek, w formie kodu.

In [46]:
```
confusion_matrix(test_labels, predictions)
```

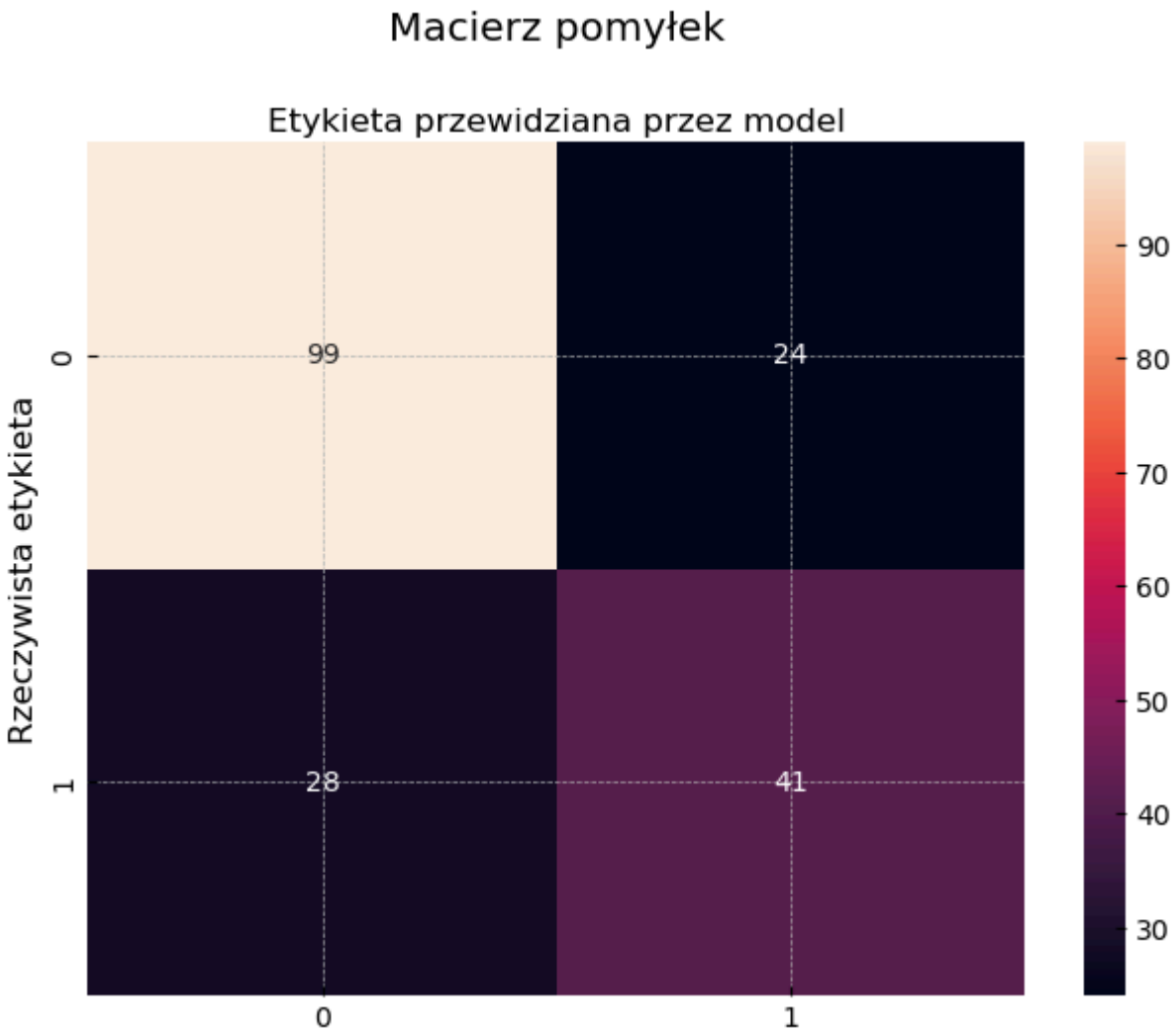
Out[46]:
```
array([[99, 24],
       [28, 41]])
```

Oraz w znanej z wcześniejszego przykładu formie graficznej.

In [47]:
```
import seaborn as sns
import matplotlib.pyplot as plt
#Tworzenie macierzy pomyłek dla testowanego klasfikatora
cnf_matrix = confusion_matrix(test_labels, predictions)
cnf_matrix
class_names=[0,1] # nazwy klas
```

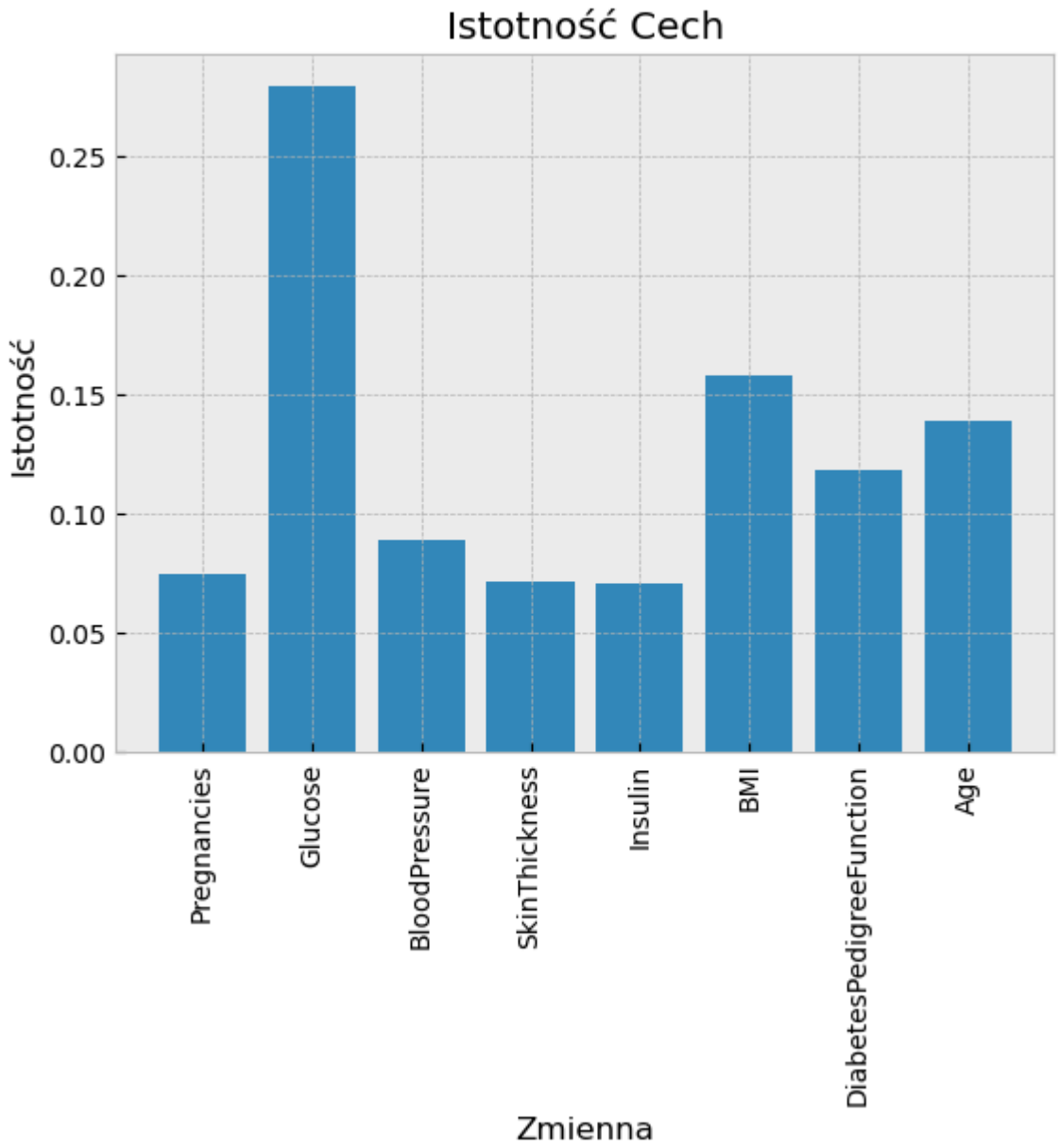
```python
fig, ax = plt.subplots()
tick_marks = np.arange(len(class_names))
plt.xticks(tick_marks, class_names)
plt.yticks(tick_marks, class_names)
# tworzenie mapy cieplnej
sns.heatmap(pd.DataFrame(cnf_matrix), annot=True ,fmt='g')
ax.xaxis.set_label_position("top")
plt.tight_layout()
plt.title('Macierz pomyłek', y=1.1)
plt.ylabel('Rzeczywista etykieta')
plt.xlabel('Etykieta przewidziana przez model')
```

Out[47]: Text(0.5, 427.9555555555555, 'Etykieta przewidziana przez model')



Dodatkowo, można wykreślić wykres istotności cech z wykorzystaniem biblioteki matplotlib.

In [48]:
```
import argparse
parser = argparse.ArgumentParser()
parser.parse_args([])

import matplotlib.pyplot as plt

plt.style.use('bmh')
x_values = list(range(len(importances)))# Tworzenie wykresu słupkowego
plt.bar(x_values, importances, orientation = 'vertical')# Podpisy osi dla osi X
plt.xticks(x_values, feature_list, rotation='vertical')
plt.ylabel('Istotność');
plt.xlabel('Zmienna');
plt.title('Istotność Cech');
```



Zadanie 1

Korzystając z pokazanych przykładów dopisać funkcje raportujące wyniki metryk i porównać z tymi w przykładzie. Wykorzystaj funkcję `classification_report` w jednej z pierwszych komórek, gdzie ładowano dataset Iris i wykorzystano tylko miarę accuracy.

Zadanie 2

Korzystając z innych parametrów niż *max_depth*, sprawdzić ich wpływ działania na klasyfikator w wybranym z przedstawionych w przykładzie datasetacie (Preferowany Diabetes). Sprawdzić parametry:

> *min_samples_split* - odpowiadający za minimalną liczbe próbek znajdujących się w węźle zanim zostanie podzielony

> *min_samples_leaf* - minimalna liczbę próbek znajdujących się w liściu

> *max_leaf_nodes* - maksymalna liczba liści

> *max_features* - maksymalna liczba cech stosowana do dzielenia węzłów

Zadanie 3

Zamienić domyślny wskaźnik zanieczyszczenia *gini* na *entropy* dla wybranego klasyfikatora w zbiorze Diabetes. Czy zaobserowano znaczącą różnicę w działaniu klasyfikatora?

Zadanie 4

Wykorzystując algorytm istotnośći cech wytrenować klasyfikator zawierający:

- 4 najlepsze cechy
- 2 najgorsze cechy

Jak zmiana tych parametów wpłyneła na metryki? Wykonaj zadanie dla zbioru Diabetes.

In [49]:
```
from sklearn.metrics import classification_report

print("ZADANIE 1\n")

print("Oceny dla zbioru Iris")
X_iris = datasets.load_iris().data
y_iris = datasets.load_iris().target
X_tr_ir, X_te_ir, y_tr_ir, y_te_ir = train_test_split(X_iris, y_iris, test_size=0.10, random_state=109)
clf_iris = DecisionTreeClassifier(max_depth=2)
clf_iris.fit(X_tr_ir, y_tr_ir)
y_pred_ir = clf_iris.predict(X_te_ir)

print("Classification report (Iris):")
print(classification_report(y_te_ir, y_pred_ir))
```

```
ZADANIE 1

Oceny dla zbioru Iris
Classification report (Iris):
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         5
           1       1.00      1.00      1.00         4
           2       1.00      1.00      1.00         6

    accuracy                           1.00        15
   macro avg       1.00      1.00      1.00        15
weighted avg       1.00      1.00      1.00        15
```

Komentarz do Zadania 1 Dodałem classification report - precision, recall, f1-score.

CO ciekawe, wytrenowane drzewo w ogóle się nie myli (wszędzie 100%). Natomiast też trzeba patrzeć na support, czyli liczbę przykładów - a jest ich mało, co nie zapewnia że model naprawdę jest taki dobry.

In [50]:
```python
print("ZADANIE 2")
print("Testowanie różnych parametrów drzewa (Diabetes)\n")

dt_default = DecisionTreeClassifier(random_state=109)
dt_default.fit(train_features, train_labels)
acc_def = accuracy_score(test_labels, dt_default.predict(test_features))
print(f"Domyślne drzewo (bez ograniczeń) Accuracy: {acc_def:.4f}\n")

params_to_test = {
    'min_samples_split': [2, 10, 50, 100],
    'min_samples_leaf': [1, 5, 20, 50],
    'max_leaf_nodes': [None, 5, 10, 20],
    'max_features': [None, 'sqrt', 'log2', 3]
}

for param_name, values in params_to_test.items():
    print(f"Testujemy parametr: {param_name}")
    for val in values:
        kwargs = {param_name: val, 'random_state': 109}
        dt = DecisionTreeClassifier(**kwargs)
        dt.fit(train_features, train_labels)
        acc = accuracy_score(test_labels, dt.predict(test_features))
        print(f"  {param_name}={val} -> Accuracy: {acc:.4f}")
    print("-" * 30)
```

```
ZADANIE 2
Testowanie różnych parametrów drzewa (Diabetes)

Domyślne drzewo (bez ograniczeń) Accuracy: 0.6875

Testujemy parametr: min_samples_split
  min_samples_split=2 -> Accuracy: 0.6875
  min_samples_split=10 -> Accuracy: 0.6979
  min_samples_split=50 -> Accuracy: 0.7240
  min_samples_split=100 -> Accuracy: 0.7396
------------------------------
Testujemy parametr: min_samples_leaf
  min_samples_leaf=1 -> Accuracy: 0.6875
  min_samples_leaf=5 -> Accuracy: 0.6979
  min_samples_leaf=20 -> Accuracy: 0.7188
  min_samples_leaf=50 -> Accuracy: 0.7500
------------------------------
Testujemy parametr: max_leaf_nodes
  max_leaf_nodes=None -> Accuracy: 0.6875
  max_leaf_nodes=5 -> Accuracy: 0.6927
  max_leaf_nodes=10 -> Accuracy: 0.7500
  max_leaf_nodes=20 -> Accuracy: 0.7344
------------------------------
Testujemy parametr: max_features
  max_features=None -> Accuracy: 0.6875
  max_features=sqrt -> Accuracy: 0.7083
  max_features=log2 -> Accuracy: 0.7031
  max_features=3 -> Accuracy: 0.7031
------------------------------
```

Komentarz do Zadania 2:

Model domyślny mocno się przeucza - zbyt rozbudowane drzewo. Przez to ma słaby wynik na zbiorze testowym - accuracy około 68.7%. Dodanie ograniczeń regularyzujących pomaga. Widać tendencję: im bardziej podnosimy węzłom poprzeczkę z `min_samples_split` czy `min_samples_leaf`, tym sprawniej drzewo generalizuje - wyniki rosną w stronę 74-75%. Nawet tryb `max_leaf_nodes=10` uzyskuje 75%. Ograniczenie ilości cech do wyboru (max_features) też trochę pomogło.

In [51]:
```python
print("ZADANIE 3: Gini a Entropy (Diabetes)\n")

dt_gini = DecisionTreeClassifier(criterion='gini', random_state=109)
dt_gini.fit(train_features, train_labels)
acc_gini = accuracy_score(test_labels, dt_gini.predict(test_features))

dt_entropy = DecisionTreeClassifier(criterion='entropy', random_state=109)
dt_entropy.fit(train_features, train_labels)
```

```
acc_entropy = accuracy_score(test_labels, dt_entropy.predict(test_features))

print(f"Accuracy ('gini'): {acc_gini:.4f}")
print(f"Accuracy ('entropy'): {acc_entropy:.4f}")
```

```
ZADANIE 3: Gini a Entropy (Diabetes)

Accuracy ('gini'): 0.6875
Accuracy ('entropy'): 0.6979
```

Komentarz do Zadania 3:

Po wymianie wskaźnika z domyślnego *Gini* na entropię, dokładność nieznacznie się polepszyła - z 68.7% do 69.8%. Różnica nie jest znacząca, ale pokazuje, że wybór miary zanieczyszczenia węzła potrafi zmodyfikować strukturę rozgałęzień na korzyść konkretnego zbioru danych.

In [55]:

```
import numpy as np

print("ZADANIE 4: Selekcja na najistotniejsze lub najgorsze cechy (Diabetes)\n")

dt_full = DecisionTreeClassifier(random_state=109)
dt_full.fit(train_features, train_labels)

feature_importances = dt_full.feature_importances_
print("Wszystkie wagi:", list(zip(feature_list, np.round(feature_importances, 4))))

sorted_indices = np.argsort(feature_importances)

best_4_indices = sorted_indices[-4:]
best_4_names = np.array(feature_list)[best_4_indices]

worst_2_indices = sorted_indices[:2]
worst_2_names = np.array(feature_list)[worst_2_indices]

print("\nNajlepsze 4 cechy: ", list(best_4_names))
print("Najgorsze 2 cechy: ", list(worst_2_names))

def evaluate_subset(X_tr, X_te, y_tr, y_te, feature_indices, desc="Subset"):
    X_tr_sub = np.array(X_tr)[:, feature_indices]
    X_te_sub = np.array(X_te)[:, feature_indices]

    dt = DecisionTreeClassifier(random_state=109)
    dt.fit(X_tr_sub, y_tr)

    y_pred = dt.predict(X_te_sub)
    acc = accuracy_score(y_te, y_pred)
    print(f"\n[{desc}] Accuracy: {acc:.4f}")
```

```python
print("\n--- Porównanie ---")
evaluate_subset(train_features, test_features, train_labels, test_labels, np.arange(len(feature_list)), "Wszystkie cechy")

evaluate_subset(train_features, test_features, train_labels, test_labels, best_4_indices, "Top 4 najlepsze cechy")

evaluate_subset(train_features, test_features, train_labels, test_labels, worst_2_indices, "Tylko 2 najgorsze cechy")
```

```
ZADANIE 4: Selekcja na najistotniejsze lub najgorsze cechy (Diabetes)

Wszystkie wagi: [('Pregnancies', np.float64(0.0687)), ('Glucose', np.float64(0.3402)), ('BloodPressure', np.float64(0.0851)), ('SkinThicknes
s', np.float64(0.0551)), ('Insulin', np.float64(0.0648)), ('BMI', np.float64(0.1576)), ('DiabetesPedigreeFunction', np.float64(0.0917)), ('A
ge', np.float64(0.1368))]

Najlepsze 4 cechy:  [np.str_('DiabetesPedigreeFunction'), np.str_('Age'), np.str_('BMI'), np.str_('Glucose')]
Najgorsze 2 cechy:  [np.str_('SkinThickness'), np.str_('Insulin')]

--- Porównanie ---

[Wszystkie cechy] Accuracy: 0.6875

[Top 4 najlepsze cechy] Accuracy: 0.6719

[Tylko 2 najgorsze cechy] Accuracy: 0.6198
```

Komentarz do Zadania 4:

Model wyuczony tylko i wyłącznie na 4 decyzyjnie najważniejszych kolumnach (`Glucose` , `BMI` , `Age` i `DiabetesPedigreeFunction`) osiągnął wynik ~67%, blisko wyniku wariantu ze wszystkimi cechami (ten jest o ~1.5% lepszy). Oznacza to, że za pomocą mniejszej porcji pamięci i obliczeń mielibyśmy bardzo zbliżoną diagnostykę. Natomiast model bazujący jedynie na dwóch parametrach wnoszących najmniej dokładność spada do 62%.